

SE 352/452: Object-Oriented Enterprise Computing

Project Overview: Spring Boot Security-Based Web Application

This project is a secure, full-stack Spring Boot web application featuring user registration, login, role-based access control, JWT-based authentication, profile management, and administrative oversight. The application integrates Spring Security, Thymeleaf, JPA/Hibernate, and static resources to create a dynamic, responsive, and well-structured system suitable for educational and real-world scenarios.

Project Packages:

1. config

- Defines security and web configuration.
- Key files: `SecurityConfig.java`, `CustomAccessDeniedHandler.java`, `CustomAuthenticationEntryPoint.java`, `WebConfig.java`
- Purpose: Sets up HTTP security rules, login/logout flow, exception handling, and resource access.

2. controllers

- Contains MVC controllers like `UserAccountController.java`.
- Purpose: Handles routes related to login, registration, profile viewing/editing, and account management.

3. dtos

- Includes `LoginRequest`, `JwtResponse`, and `ApiExceptionDto`.
- Purpose: Transfers structured data between frontend, backend, and external clients, especially for authentication and error handling.

4. entities

- JPA models like `User.java` and `Role.java`.
- Purpose: Represents database tables and relationships; supports role-based access control.

5. initializers

- Contains `DataInitializer.java`.
- Purpose: Seeds the database with default users and roles at application startup for dev/testing. Roles include USER, ADMIN, and MANAGER.

6. jwt

- Contains `JwtUtil.java`.
- Purpose: Handles creation, validation, and parsing of JWTs used for stateless authentication.

7. repositories

- Includes `UserRepository` and `RoleRepository`.
- Purpose: Provides CRUD access to users and roles using Spring Data JPA.

8. services

- Includes interfaces and implementations for `AuthService` and `CustomUserDetailsService`.
- Purpose: Encapsulates business logic related to authentication and user data retrieval.

9. utils

- Contains `FileUploadUtil.java`.
- Purpose: Handles file I/O logic such as saving and deleting user profile pictures.

10. static resources

- Files in `static/` like `login.css` and the `images/` folder.
- Purpose: Hosts frontend styling and media assets served directly to clients.

11. templates

- Thymeleaf templates: `login.html`, `register.html`, `profile.html`, `all_users.html`, `dashboard.html`, etc.
- Purpose: Renders dynamic HTML views for user interaction based on model data.

Application Features:

- User Registration & Login
- JWT and Form-based Authentication
- Role-Based Access (USER, ADMIN, MANAGER)
- Profile Picture Uploads
- Static Resource Handling
- Admin Dashboard for Viewing All Users

This structure promotes modularity, testability, and separation of concerns while demonstrating best practices in modern Spring Boot web development.

Project Package & Class Details:

Package: edu.hield.security.config

This package contains all configuration components necessary to control the behavior of both Spring Security and Spring MVC. These classes ensure the application has well-defined security policies, proper error handling, and appropriate routing for both API and web-based access.

1. SecurityConfig.java

- **Purpose:** Acts as the primary class for setting up Spring Security.
- **What It Does:**
 - Declares a `SecurityFilterChain` bean that defines access rules for various URL patterns.
 - Configures which endpoints are public (e.g., "/", "/login", "/register", static resources) and which require authentication.
 - Requires authentication for all other endpoints.
 - Enables HTTP basic and form-based login.
 - Disables CSRF for simplicity and to allow easier API testing.
 - Configures stateless session management, suitable for REST APIs or JWT-based authentication.
 - Sets custom handlers for:
 - **Unauthorized Access** (handled by `CustomAuthenticationEntryPoint`) and
 - **Forbidden Access** (handled by `CustomAccessDeniedHandler`).
 - Allows Spring method-level security annotations like `@PreAuthorize`.
- **Why It's There:** This class is the centralized point for enforcing how users are authenticated and what they're allowed to access. It supports clean separation between public, user, and admin roles.

2. CustomAccessDeniedHandler.java

- **Purpose:** Handles 403 Forbidden errors when a logged-in user tries to access a resource they don't have permission to access.
- **What It Does:**
 - Implements Spring Security's `AccessDeniedHandler` interface.
 - Logs the access denial with user details and the requested URI.
 - Sends a JSON response with status 403 and an error message.

- **Why It's There:** By default, Spring returns a generic error page or message. This class makes it possible to send a clean, user-readable message or JSON for API clients when access is denied.

3. CustomAuthenticationEntryPoint.java

- **Purpose:** Handles 401 Unauthorized responses for unauthenticated users.
- **What It Does:**
 - Implements Spring Security's `AuthenticationEntryPoint` interface.
 - Triggered when someone tries to access a secured endpoint without being logged in.
 - Logs the unauthorized attempt.
 - Sends a JSON response with status 401 and a descriptive error.
- **Why It's There:** Provides clean and consistent error messages for unauthenticated access attempts, especially useful for REST APIs or SPAs that need structured error responses.

4. WebConfig.java

- **Purpose:** Configures MVC-specific settings that are outside the scope of Spring Security.
- **What It Does:**
 - Implements `WebMvcConfigurer` to customize Spring MVC behavior.
 - May include resource handlers for serving static content such as `/css/**`, `/js/**`, `/images/**`.
 - Used to define any additional view resolvers, formatters, or interceptors.
- **Why It's There:** Separates general web configuration from security concerns. Ensures the UI behaves as expected and that static resources are properly exposed in the application.

Package: edu.hield.security.controllers

This package contains the web controller responsible for handling user-related UI actions and serving web pages using Thymeleaf templates. It acts as the bridge between the front-end (user interface) and the application's backend logic.

1. UserAccountController.java

- **Purpose:** Manages account-related web endpoints such as login, registration, profile editing, and account display.
- **What It Does:**
 - Uses `@Controller` and `@RequestMapping("/account")` to designate it as a Spring MVC controller handling all requests under the `/account` path.

- **GET /account/login:** Serves the login form view.
- **GET /account/register:** Serves the user registration form.
- **POST /account/register:** Handles the submission of new user registration data.
- **GET /account/profile:** Displays the current user's profile.
- **POST /account/profile:** Allows users to update their profile, including uploading a new profile picture.
- Integrates with `UserService` to persist and retrieve user data.
- Uses `Principal` to identify the currently authenticated user for profile operations.
- **Why It's There:** This controller handles all interactions that users have related to their account. It connects the web views (login, register, profile) with the underlying services, ensuring proper data flow and secure interaction.
- **Security Context:**
 - Access to endpoints is protected by Spring Security configurations.
 - Authenticated users can access profile-related endpoints.
 - Anonymous users can access login and register pages.
- **Template Views:**
 - Returns names of Thymeleaf template pages (e.g., `account/login.html`, `account/profile.html`).

This controller serves as the entry point for user interactions, encapsulating all routing logic and template bindings needed to provide a secure, interactive UI for user account management.

Package: `edu.hield.security.dtos`

This package contains Data Transfer Objects (DTOs), which are lightweight objects used to encapsulate data exchanged between the client, controller, and service layers. DTOs are not persisted to the database and are used to structure API input/output cleanly.

1. `LoginRequest.java`

- **Purpose:** Carries login credentials from the client to the backend.
- **Fields:**
 - `String username:` The user's login identifier.
 - `String password:` The user's password.
- **What It Does:** Used in login forms and authentication API calls to encapsulate user credentials.
- **Why It's There:** Separates login data from entity models, ensuring encapsulation and validation in authentication workflows.

2. JwtResponse.java

- **Purpose:** Represents the server's response to a successful authentication request.
- **Fields:**
 - `String token`: The JWT access token.
 - `String username`: The authenticated user's username.
 - `List<String> roles`: The user's assigned roles.
- **What It Does:** Returned to the client after login to provide access credentials for subsequent requests.
- **Why It's There:** Ensures clear communication of authentication success and user role information. Keeps token logic separate from user entities.

3. ApiExceptionDto.java

- **Purpose:** Structures error messages returned to the client.
- **Fields:**
 - `String error`: A short error code or label.
 - `String message`: A descriptive error message.
- **What It Does:** Used in exception handlers to return consistent, readable error responses.
- **Why It's There:** Promotes standardization of error formats in REST APIs and improves client-side error handling.

These DTOs are key to ensuring clean API interfaces, separating internal models from external communication formats, and reducing the risk of data overexposure.

Package: edu.hield.security.entities

This package contains JPA entity classes that directly map to the application's database tables. These classes represent core domain models such as users and roles, and they define how data is persisted, queried, and managed by Spring Data JPA.

1. User.java

- **Purpose:** Represents a registered user in the system.
- **Annotations & Structure:**
 - Annotated with `@Entity` to indicate it's a JPA-managed table.
 - Includes `@Id` and `@GeneratedValue` for unique primary key generation.

- **Fields:**
 - Long id: Primary key.
 - String username: Unique identifier for login (annotated with `@Column(unique = true)`).
 - String password: Encrypted password.
 - String fullName: Full name of the user.
 - String profilePicture: Filename of the user's uploaded profile image.
 - Set<Role> roles: Many-to-many relationship with roles, managed by `@ManyToMany` and `@JoinTable`.
- **What It Does:**
 - Defines the structure of a user in the database.
 - Supports role-based access control via linked roles.
- **Why It's There:** Serves as the main authenticated principal in the system, integrating with Spring Security for authentication and authorization.

2. Role.java

- **Purpose:** Represents a user role or authority (e.g., ROLE_USER, ROLE_ADMIN, ROLE_MANAGER).
- **Annotations & Structure:**
 - Annotated with `@Entity`.
 - Uses `@Id` and `@GeneratedValue` for automatic ID generation.
- **Fields:**
 - Long id: Primary key.
 - String name: The name of the role (e.g., "ROLE_USER").
 - Set<User> users: Mapped from the other side of the many-to-many relationship.
- **What It Does:**
 - Enables assigning multiple roles to users.
 - Used by Spring Security to determine access permissions.
- **Why It's There:** Provides a flexible role-based authorization model allowing users to have varying levels of access.

These entity classes are core to the security model. They define the structure of users and their roles, and enable persistence, lookups, and enforcement of access controls in the system.

Package: edu.hield.security.initializers

This package contains application bootstrap logic that runs at startup. Initializers are typically used to prepopulate the database with default data or configuration, which is especially useful for development and testing environments.

1. DataInitializer.java

- **Purpose:** Automatically sets up essential data when the application starts.
- **How It Works:**
 - Annotated with `@Component`, allowing Spring to detect and manage it as a bean.
 - Contains a method annotated with `@PostConstruct`, which tells Spring to run this logic once the bean is fully initialized.
- **What It Does:**
 - Creates default roles (e.g., `ROLE_USER`, `ROLE_ADMIN`, `ROLE_ADMIN`) if they don't already exist in the database.
 - Creates a few default users and assigns them roles.
 - Encodes passwords using a `PasswordEncoder` before storing them.
- **Why It's There:**
 - Ensures the system is in a usable state after startup.
 - Avoids the need to manually create users or roles via SQL or UI in development.
 - Helps quickly test security features like login and role-based access.

This class is a practical way to seed the application with meaningful test data without requiring external scripts or database migrations. It ensures consistency across test environments and streamlines the initial setup process.

Package: edu.hield.security.jwt

This package contains utility classes responsible for handling JWT (JSON Web Token) operations. JWTs are used for stateless authentication—allowing secure transmission of user identity and roles without maintaining session state on the server.

1. JwtUtil.java

- **Purpose:** Centralized utility for generating, parsing, and validating JWT tokens.

- **Core Responsibilities:**
 - **Token Creation:** Generates JWTs for authenticated users using a secret key, embedding claims such as username and user roles.
 - **Token Parsing:** Extracts claims from an incoming JWT, such as the subject (username), issue date, expiration date, and custom roles.
 - **Validation:** Checks if the token is properly signed, unexpired, and associated with the correct user.
- **Key Components:**
 - Uses a secret key (typically injected from configuration) to sign tokens with HMAC SHA-256.
 - Parses tokens using the io.jsonwebtoken (JJWT) library.
 - Includes helper methods like:
 - `generateToken(UserDetails userDetails)`
 - `validateToken(String token, UserDetails userDetails)`
 - `extractUsername(String token)`
- **Why It's There:** JWTs are critical for stateless security in modern REST APIs. This class isolates all token-related logic, making the security layer modular and maintainable.

The `JwtUtil` class is essential for secure, stateless authentication. It ensures that only properly signed tokens with valid content are accepted, and that they carry the necessary claims to authorize the user's actions.

Package: `edu.hield.security.repositories`

This package contains Spring Data JPA repository interfaces for interacting with the application's database. These interfaces abstract away low-level data access code and provide built-in CRUD operations and query generation.

1. `UserRepository.java`

- **Purpose:** Provides database access methods for `User` entities.
- **Extends:** `JpaRepository<User, Long>`
- **Key Methods:**
 - `Optional<User> findByUsername(String username)`: Fetches a user by their unique username.
 - `List<User> findAllByOrderByLastNameAsc()`: Retrieves all users sorted alphabetically by last name.
- **What It Does:**

- Enables Spring to automatically implement common database queries.
 - Supports user lookups needed for authentication and profile display.
- **Why It's There:** Central to managing user persistence, enabling login functionality, profile updates, and admin operations.

2. RoleRepository.java

- **Purpose:** Manages access to `Role` entities in the database.
- **Extends:** `JpaRepository<Role, Long>`
- **Key Methods:**
 - `Optional<Role> findByName(String name):` Retrieves a role by its name (e.g., "ROLE_USER").
- **What It Does:**
 - Facilitates role lookup during user registration or initialization.
 - Assists in assigning appropriate permissions to users.
- **Why It's There:** Enables efficient role management and integration with Spring Security's authorization system.

These repositories simplify data operations by relying on Spring Data's powerful query derivation and interface-based design. They allow other parts of the application (e.g., services and controllers) to work with persistent data cleanly and efficiently.

Package: edu.hield.security.services

This package defines and implements the application's business logic related to authentication and user management. Services act as intermediaries between controllers and repositories, encapsulating logic and enforcing security or validation rules.

1. AuthService.java

- **Purpose:** Defines the contract for authentication operations.
- **Type:** Java interface.
- **Method:**
 - `JwtResponse authenticateAndGenerateToken(LoginRequest request)`
- **What It Does:**
 - Declares a method for verifying login credentials and issuing JWTs.
- **Why It's There:** Provides a clear abstraction for login logic. Allows flexible implementation, easier testing, and decoupling from specific frameworks.

2. AuthServiceImpl.java

- **Purpose:** Implements the authentication logic defined in `AuthService`.
- **What It Does:**
 - Authenticates users using the `AuthenticationManager`.
 - On success, sets the security context.
 - Generates and returns a `JwtResponse` using `JwtUtil`.
 - Handles `AuthenticationException` to detect and report login failures.
- **Why It's There:** Provides the concrete implementation of login logic, ensuring the authentication flow is secure and encapsulated.

3. CustomUserDetailsService.java

- **Purpose:** Declares a method to retrieve a user's details by username, used by Spring Security.
- **Type:** Java interface extending `UserDetailsService`.
- **What It Does:**
 - Forces any implementation to provide a method: `UserDetails loadUserByUsername(String username)`.
- **Why It's There:** Required by Spring Security to authenticate users against custom `User` entities.

4. CustomUserDetailsServiceImpl.java

- **Purpose:** Loads user-specific data to support Spring Security authentication.
- **What It Does:**
 - Implements `loadUserByUsername` using the `UserRepository`.
 - Converts `User` entities to Spring Security's `UserDetails` objects.
 - Maps roles to `GrantedAuthority` objects.
 - Throws `UsernameNotFoundException` if user lookup fails.
- **Why It's There:** Integrates the custom `User` entity with Spring Security's internal authentication mechanisms.

These services are critical to the application's authentication and user management system. They abstract business logic, reduce repetition in controllers, and facilitate secure, role-based access handling.

Package: edu.hield.security.utils

This package contains utility classes that offer common, reusable operations across the application. Utility classes improve maintainability by isolating logic that doesn't belong to any specific business or controller layer.

1. FileUploadUtil.java

- **Purpose:** Provides file-related helper methods to support uploading, storing, and deleting files.
- **What It Does:**
 - Handles saving uploaded profile pictures to a local directory.
 - Ensures directories exist before writing files.
 - Optionally deletes old profile images when a new one is uploaded.
 - Deals with file naming conventions and I/O safety.
- **Key Methods:**
 - `saveFile(String uploadDir, String fileName, MultipartFile multipartFile)`
 - `deleteFileIfExists(String filePath)`
- **Why It's There:** File handling can be complex and error-prone. This utility class ensures clean, consistent logic for managing user-uploaded profile images, keeping the controller and service layers focused on business logic.

Utility classes like this are often stateless and static, making them easy to test and share across different packages without introducing tight coupling.

Folder: src/main/resources/static

This folder contains static web assets—files that are served directly to the client without being processed by the server. These are typically referenced in Thymeleaf templates or HTML pages and include CSS stylesheets, JavaScript files, and images.

1. login.css

- **Purpose:** Provides custom CSS styling for the login and registration pages.
- **What It Does:**
 - Defines the layout, fonts, spacing, and colors used on login-related forms.
 - May also apply styling for error messages, input validation, and overall branding.

- **Why It's There:**
 - Enhances user experience by making the login and registration pages visually appealing and easier to use.
 - Keeps styling modular and separate from the HTML templates for maintainability.

2. images/

- **Purpose:** Stores user-uploaded profile pictures and default image assets.
- **What It Does:**
 - Holds PNG, JPG, or other formats of user profile pictures.
 - Contains fallback/default images used when users haven't uploaded a profile picture.
- **Why It's There:**
 - Makes it easy to retrieve and display profile images via URL (e.g., `/images/username.png`).
 - Keeps media assets centralized and publicly accessible via Spring's static resource handling.

Usage Notes:

- Files in this directory are served automatically by Spring Boot.
- Images and CSS are typically referenced in HTML with the Thymeleaf `@{}` syntax (e.g., `@{/images/default.png}`).

Static resources like these help provide a polished, responsive front-end without requiring server-side logic to render them. They are essential to modern web application user interfaces.

Folder: `src/main/resources/templates`

This folder contains Thymeleaf template files that make up the server-rendered HTML views of the application. These templates are dynamically processed by Spring MVC controllers and serve as the primary way users interact with the application via a browser. They include pages for authentication, user profiles, dashboards, team management, and administrative functions. Thymeleaf syntax enables seamless integration with backend data and allows conditional rendering, iteration, and form binding directly within the HTML structure.

1. `login.html`

- **Purpose:** Provides a secure and styled interface for users to log in.

- **What It Does:**
 - Renders a login form bound to Spring Security.
 - Displays error messages when authentication fails.
 - Links to registration and password recovery if needed.
- **Why It's There:** Facilitates user authentication using form-based login, working in conjunction with Spring Security.

2. register.html

- **Purpose:** Allows new users to create an account.
- **What It Does:**
 - Renders a registration form with fields such as username, password, and full name.
 - Validates input and posts form data to the registration endpoint.
- **Why It's There:** Enables a clean and user-friendly onboarding process for new users.

3. profile.html

- **Purpose:** Displays and allows updates to the authenticated user's profile.
- **What It Does:**
 - Shows user details like full name, username, and profile picture.
 - Provides a form to update user information and upload a new profile image.
- **Why It's There:** Gives users control over their personal account information and visuals.

4. all_users.html

- **Purpose:** Provides an administrative interface to view all registered users.
- **What It Does:**
 - Displays a table of users including usernames, roles, and profile pictures.
 - Uses Thymeleaf to iterate through the list and format the content dynamically.
- **Why It's There:** Helps admins oversee user activity and manage system roles.

5. account_settings.html

- **Purpose:** Offers advanced configuration options for user accounts.
- **What It Does:**
 - Displays settings such as email preferences, password changes, or display options.
 - Allows users to customize aspects of their experience.
- **Why It's There:** Supports user autonomy and personalization beyond basic profile fields.

6. my_team.html

- **Purpose:** Displays team-related data for users who are part of a team or group.
- **What It Does:**
 - Likely shows team members, roles, or statuses.
 - May allow the user to initiate or manage collaborative actions.
- **Why It's There:** Enhances collaborative features and promotes social aspects of the app.

7. dashboard.html

- **Purpose:** Serves as the main control center for a logged-in user.
- **What It Does:**
 - Presents navigation to other parts of the app like profile, team, or admin tools.
 - May show a personalized summary or quick-access widgets.
- **Why It's There:** Provides a clear starting point for user activity post-login.

8. index.html

- **Purpose:** Acts as the public-facing homepage of the application.
- **What It Does:**
 - Displays general information about the app.
 - Offers links to login or register.
- **Why It's There:** Serves as the app's entry point for unauthenticated users and sets the tone for user engagement.

Together, these templates form the user interface layer of the application. They bridge the backend services with a polished front-end experience and support secure, personalized, and role-based views for all user types.